

StayFlow

Relatório de Testes de Software API e Interface

Arquivo analisado: StayFlow-main.zip

Data: 21/05/2026

Resumo executivo: o backend original não inicializou por erro no modelo Booking. O frontend apresentou falhas de lint, TypeScript e build. Em uma cópia temporária corrigida apenas para exploração, a suíte de API executou 15 cenários: 13 passaram e 2 falharam.

1. Escopo e metodologia

Foram realizados testes técnicos sobre o ZIP enviado, cobrindo backend Django REST Framework e frontend Next.js. O repositório original não foi alterado. Para investigar além do primeiro bloqueador do backend, usei uma cópia temporária com banco SQLite e correções mínimas de ambiente.

Área	Procedimento	Resultado
Backend - dependências	pip install a partir de requirements.txt	FAIL: versões inexistentes e pacote ausente
Backend - inicialização	python manage.py check	FAIL: TypeError no modelo Booking
API exploratória	DRF APIClient em cópia temporária	13 PASS / 2 FAIL
Frontend - dependências	npm install	PASS com ressalvas: 3 vulnerabilidades
Frontend - lint	npm run lint	FAIL: 4 erros, 7 warnings
Frontend - TypeScript	npx tsc --noEmit	FAIL: múltiplos erros
Frontend - build	npm run build	FAIL: Google Font externa indisponível

2. Bloqueadores principais

BLOQ-01 - Crítico - Backend não inicia

- Evidência: apps/bookings/models.py define o campo property na linha 17 e usa @property na linha 76. Dentro da classe, o nome property passa a apontar para o ForeignKey, causando TypeError: ForeignKey object is not callable.
- Impacto: A API fica indisponível e os endpoints não podem ser testados no projeto original.
- Recomendação: Renomear o campo para listing/property_obj ou usar builtins.property temporariamente. A renomeação é a opção mais limpa.

BLOQ-02 - Alto - requirements.txt não é instalável

- Evidência: back/requirements.txt usa PyJWT==2.8.1 e django-rest-framework-simplejwt==5.3.2, versões não encontradas; também falta django-filter, embora seja importado nas views.
- Impacto: Ambiente novo não sobe de forma reproduzível.
- Recomendação: Corrigir as versões e adicionar django-filter ao requirements.txt.

BLOQ-03 - Crítico - User customizado não configurado

- Evidência: Existe apps.users.models.User herdando AbstractUser, mas settings.py não define AUTH_USER_MODEL = "users.User".
- Impacto: Autenticação, migrações e relações do domínio ficam inconsistentes.
- Recomendação: Definir AUTH_USER_MODEL antes de consolidar as migrações e revisar o fluxo de login.

BLOQ-04 - Alto - Frontend ainda depende de Supabase legado

- Evidência: bookings/page.tsx, properties/new/page.tsx e useAuth.ts ainda usam supabase.from ou supabase não importado, enquanto lib/supabase.ts está deprecated e não implementa from().
- Impacto: Dashboard, criação de imóvel e reservas tendem a quebrar em runtime/compilação.
- Recomendação: Migrar todos os fluxos ativos para front/lib/api.ts.

BLOQ-05 - Médio - Build depende de rede para fonte externa

- Evidência: npm run build falhou ao tentar baixar Poppins via next/font/google.
- Impacto: Deploy pode falhar em ambiente sem acesso externo.
- Recomendação: Usar fonte local, fallback CSS ou configuração de build sem fetch externo.

3. Resultado dos testes de API

Observação: a suíte abaixo foi executada em cópia temporária após contornar o bloqueador de importação e configurar banco de teste. Ela não representa que o projeto original sobe sem correções; serve para revelar falhas funcionais posteriores.

Métrica	Valor
---------	-------

Cenários executados	15
Passaram	13
Falharam	2
Taxa de aprovação exploratória	86.7%

ID	Status	Cenário	Obtido
API-001	PASS	Listagem publica de propriedades	HTTP 200
API-002	PASS	Criacao de propriedade sem autenticao bloqueada	HTTP 401
API-003	PASS	Registro retorna usuario e tokens JWT	HTTP 201; keys=['user', 'access', 'refresh']
API-004	FAIL	Login por email e senha	HTTP 400; data={'non_field_errors': [ErrorDetail(string='Invalid email or password.', code='invalid')]}
API-005	PASS	Perfil autenticado /me	HTTP 200; data={'id': 1, 'email': 'owner@example.com', 'username': 'owner', 'first_name': 'Owner', 'last_name': 'One', '}
API-006	PASS	Criacao de propriedade autenticada	HTTP 201; data={'id': 1, 'title': 'Casa Teste', 'description': 'desc', 'property_type': 'house', 'address': 'Rua 1', 'ci
API-007	PASS	Detalhe publico de propriedade	HTTP 200
API-008	PASS	Filtro por cidade	HTTP 200; count=1

ID	Status	Cenário	Obtido
API-009	PASS	Reserva com data inicial passada rejeitada	HTTP 400; data={'non_field_errors': [ErrorDetail(string='Start date cannot be in the past.', code='invalid')]}
API-010	PASS	Criacao de reserva calcula preco total	HTTP 201; data={'id': 1, 'property': 1, 'property_title': 'Casa Teste', 'property_price': '1200.00', 'user': 1, 'user_na
API-011	PASS	Listagem de reservas do usuario	HTTP 200; len=1
API-012	PASS	Datas ocupadas antes de confirmacao nao incluem pendentes	HTTP 200; data={'property_id': '1', 'booked_dates': []}
API-013	PASS	Confirmacao de reserva pelo proprietario	HTTP 200; data={'id': 1, 'property': 1, 'property_title': 'Casa Teste', 'property_price': '1200.00', 'user': 1, 'user_na
API-014	PASS	Datas ocupadas apos confirmacao	HTTP 200; data={'property_id': '1', 'booked_dates': ['2026-05-31', '2026-06-01', '2026-06-02', '2026-06-03']}
API-015	FAIL	Usuario nao proprietario nao pode editar propriedade	HTTP 200; titulo_atual=Casa Teste

4. Falhas funcionais identificadas

BUG-01 - Crítico - Login por email falha após registro

- Evidência: API-004 retornou HTTP 400 com "Invalid email or password" para usuário recém-registrado.
- Impacto: O usuário consegue receber token no cadastro, mas não consegue autenticar depois pelo login.
- Recomendação: Se login for por email, implementar backend de autenticação por email ou ajustar username=email na criação.

BUG-02 - Crítico - Permissão de edição de propriedade retorna status incorreto

- Evidência: API-015: usuário não proprietário tentou PATCH em /properties/{id}/ e recebeu HTTP 200.
- Impacto: O cliente interpreta sucesso onde deveria haver 403; isso cria risco de segurança e inconsistência.
- Recomendação: Não retornar Response em perform_update/perform_destroy. Levantar PermissionDenied ou sobrescrever update/destroy.

BUG-03 - Alto - Cadastro do frontend envia payload incompleto

- Evidência: front/lib/api.ts envia { email, password, username }, mas RegisterSerializer exige password2.
- Impacto: Cadastro pela interface tende a retornar 400.
- Recomendação: Enviar password2 e separar nome em first_name/last_name se necessário.

BUG-04 - Alto - Reserva do frontend usa campo divergente

- Evidência: createBooking envia property_id; o serializer do backend espera property.
- Impacto: Criação de reserva pela interface tende a falhar com campo obrigatório ausente.
- Recomendação: Alterar payload para property ou aceitar ambos no serializer.

BUG-05 - Alto - Dashboard usa hook quebrado

- Evidência: hooks/useAuth.ts usa supabase.from sem importar supabase.
- Impacto: Dashboard não deve carregar propriedades corretamente.
- Recomendação: Trocar por getUserProperties() ou getProperties() de lib/api.ts.

5. Resultado dos testes de interface/frontend

Teste	Resultado	Evidência
UI-001 npm install	PASS com ressalvas	Instalação concluída; npm audit reportou 2 vulnerabilidades moderadas e 1 alta.
UI-002 npm run lint	FAIL	4 erros e 7 warnings: any explícito, setState em effect, headers e imports/imagens.
UI-003 TypeScript	FAIL	Erros de Supabase legado, @supabase/supabase-js ausente, supabase não importado, Framer Motion variants e HeadersInit.
UI-004 Build	FAIL	Falhou ao baixar Poppins de fonts.googleapis.com via next/font/google.
UI-005 Revisão dos fluxos	FAIL parcial	Login usa API Django, mas reservas e cadastro de imóvel ainda usam Supabase legado.

6. Recomendações priorizadas

Prioridade	Ação
P0	Corrigir backend para iniciar: Booking.property/@property, versões do requirements, django-filter e AUTH_USER_MODEL.
P0	Corrigir autenticação: decidir email vs username e ajustar LoginSerializer/backend de autenticação.
P0	Remover Supabase dos fluxos ativos e migrar dashboard, criação de imóvel e reservas para lib/api.ts.
P1	Alinhar contratos frontend-backend: register com password2; booking com property; filtros consistentes.
P1	Endurecer permissões de update/destroy com PermissionDenied ou permissões customizadas.
P1	Resolver lint, TypeScript e build com fonte local/fallback.
P2	Criar pipeline CI com manage.py check, pytest, npm lint, tsc, build, audit e testes E2E.

7. Suíte recomendada após correções

Backend/API

- Registro, login, /me e refresh/logout.
- CRUD de propriedades com proprietário e usuário não proprietário.
- Filtros por cidade, tipo, status, preço e ordenação.
- Reservas com datas válidas/inválidas, cálculo de preço, confirmação, cancelamento e datas ocupadas.

Frontend/E2E

- Login, cadastro e redirecionamento de rota protegida.
- Dashboard vazio e dashboard com propriedades.
- Cadastro de imóvel com validações obrigatórias.

- Fluxo de reserva e tratamento de erros 400/401/500.

Qualidade

- Testes em CI a cada pull request.
- Factories e banco isolado para backend.
- Mocks de API para frontend e cobertura de estados loading/error.

8. Parecer final

O StayFlow possui boa intenção arquitetural e um domínio organizado em usuários, imóveis e reservas. Porém, no estado atual, não está pronto para homologação nem deploy. A maior prioridade é estabilizar a execução do backend e remover os fluxos legados de Supabase do frontend.

Classificação geral: NÃO APROVADO PARA DEPLOY. Recomendação: corrigir P0/P1 e reexecutar a suíte regressiva completa.